POKHARA UNIVERSITY

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

LABORATORY REPORT

AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 3

BESE III YEAR - VI SEM

Submitted By:

Pranish Poudel
Roll No: 26
Program: BESE ,2023

Submitted To:

Er. Rajendra Bahadur Thapa
Lecturer

Date: July 3, 2026

NAME OF EXPERIMENT

Performing different kinds of software testing using the Jest framework.

AIM

To carry out unit testing on a JavaScript calculator program with the Jest framework, and to survey other testing tools and techniques including test case generation, JUnit, Selenium, and load testing.

THEORY

Software testing is the phase of development in which an application is checked to confirm that it works correctly, satisfies its requirements, and is free of defects. Its main purpose is to catch bugs early, raise reliability, and improve overall product quality before release.

In Agile development testing is not left until the end; it is woven throughout the process. Testing happens at several levels, unit, integration, system, and acceptance. This experiment concentrates on unit testing with Jest while also reviewing a set of other widely used testing tools.

1. Test Case Generation: This is the methodical activity of designing inputs, execution steps, and expected results to check whether a feature behaves as intended. A good test case records the test input given to the function, the expected output, and the actual result produced at runtime. Careful test case design maximises code coverage and surfaces edge cases such as division by zero or negative inputs that might otherwise slip through.

2. Jest Framework: Jest is an open-source JavaScript testing framework created by Meta. It is heavily used for testing JavaScript projects, especially those built with React and Node.js. Developers value it for being simple, fast, and feature-rich.

- Zero configuration: it works for most JavaScript projects straight away.
- Watch mode: it re-runs affected tests automatically as files change.
- Built-in assertions: intuitive matchers such as expect().toBe() verify results.
- Coverage reports: detailed reports show which parts of the code are tested.
- Snapshot testing: the rendered state of UI components can be captured and compared.

3. JUnit: JUnit is a unit-testing framework for Java. It uses annotations such as @Test, @Before, and @After to declare test methods and setup routines, and it integrates smoothly with build tools like Maven and Gradle, making it a mainstay of Java testing.

4. Selenium: Selenium is an open-source automation tool for testing web applications across browsers and platforms. Rather than testing individual functions, it drives the browser, clicking buttons, filling forms, navigating pages, and checking rendered content. It supports Java, Python, C#, and JavaScript and is central to end-to-end functional testing.

5. Load Testing: Load testing is a non-functional technique that measures how an application behaves under different volumes of users and data. Its aim is to expose performance bottlenecks, gauge response times, and confirm stability under heavy demand. Popular tools include Apache JMeter, Artillery, k6, and Locust.

OBJECTIVES

- To understand the fundamentals of software testing and why it matters.
- To implement unit testing for a JavaScript program using Jest.

- To design effective test cases for arithmetic functions such as addition, multiplication, and division.
- To review JUnit and its role in Java unit testing.
- To understand how Selenium automates browser-based testing.
- To learn the purpose of load testing and its common tools.

ALGORITHM

1. Install Node.js and npm on the system.
2. Create a project directory and initialise it with npm init -y.
3. Install Jest as a development dependency with npm install --save-dev jest.
4. Create calculator.js and implement add(), multiply(), and divide().
5. Export the functions with module.exports so tests can import them.
6. Create calculator.test.js in the same directory.
7. Import the calculator functions using require().
8. Write a test for each function using test() and expect().toBe().
9. Add an edge-case test that expects divide() to throw when the divisor is zero.
10. Add a "test": "jest" script to package.json.
11. Run the suite with npm test or npx jest.
12. Confirm that every test case passes.
13. Study JUnit, Selenium, and load testing as complementary techniques.

CODE

calculator.js

```
function add(a, b) {
    return a + b;
}

function multiply(a, b) {
    return a * b;
}

function divide(a, b) {
    if (b === 0) {
        throw new Error("Cannot divide by zero");
    }
    return a / b;
}

module.exports = { add, multiply, divide };
```

calculator.test.js

```
const { add, multiply, divide } = require('./calculator');

test('Addition Test', () => {
    expect(add(10, 5)).toBe(15);
});

```

```
test('Multiplication Test', () => {
    expect(multiply(4, 5)).toBe(20);
});

test('Division Test', () => {
    expect(divide(20, 5)).toBe(4);
});

test('Division by Zero Test', () => {
    expect(() => divide(20, 0)).toThrow('Cannot divide by zero');
});
```

Commands to run the tests

```
npm install --save-dev jest
npx jest
```

TEST CASES

Test Case	Input	Expected Output	Result
Addition	10, 5	15	Pass
Multiplication	4, 5	20	Pass
Division	20, 5	4	Pass
Division by Zero	20, 0	Error (Throw)	Pass

Table 1: Test case summary for the calculator functions.

OUTPUT

The calculator was tested successfully with Jest. The following was observed.

- The addition test passed (10 + 5 = 15).
- The multiplication test passed (4 x 5 = 20).
- The division test passed (20 / 5 = 4).
- The division-by-zero test threw the expected error message.
- Every test executed without failure, confirming the functions are correct.
- Jest printed a clear summary showing the number of passed and failed tests.

RESULT

The calculator application was unit-tested successfully with Jest. All four test cases, including the division-by-zero edge case, passed, demonstrating the correctness and robustness of the arithmetic functions. Theoretical knowledge of JUnit, Selenium, and load testing was also gained, broadening the understanding of testing approaches used in industry.

CONCLUSION

This experiment gave practical experience of software testing with Jest. Unit testing proved an effective way to verify individual functions and to catch errors early. Studying JUnit, Selenium, and load testing revealed the range of tools available for different situations, from back-end unit testing to front-end browser automation and performance evaluation. Adopting a well-rounded testing strategy is essential for

delivering reliable, high-quality software in an Agile environment.